Question 3 — Automate Application Deployment

Student: Kartik Roll No.: 2301730205

1. Aim

Automate the deployment of the Flask application so that every push to the Git repository triggers Jenkins to build, test and deploy the application without any manual step.

2. Requirements

- Jenkins with the Git and Pipeline plugins installed.
- A Poll SCM trigger configured to check the repository every minute.
- deploy.sh script with sudo permission to copy files and restart the service.
- A systemd service unit (devops-lab1.service) managing the Flask process.

3. Poll SCM Configuration

The Jenkins job was configured with a Poll SCM trigger using the cron expression * * * * * (every minute). Jenkins polls the GitHub repository and triggers a new build only when it detects a new commit.

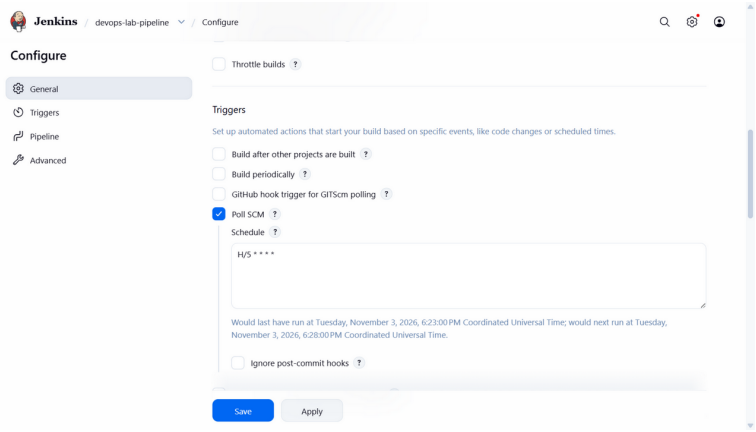


Fig: Jenkins Poll SCM trigger — every minute

4. Sudoers Configuration

The Jenkins user was granted passwordless sudo permission for the deploy script by adding the following line to /etc/sudoers.d/jenkins:

```
jenkins ALL=(ALL) NOPASSWD: /bin/bash /var/lib/jenkins/workspace/devops-lab-1/scripts/deploy.sh
```

5. deploy.sh — Deployment Script

```
#!/bin/bash
set -e

APP_DIR=/opt/devops-lab1
SERVICE_NAME=devops-lab1
WORKSPACE=${WORKSPACE:-$(pwd)}

echo '=== DevOps Lab 1 — Deployment Script ==='
echo "Workspace : $WORKSPACE"
echo "Target : $APP_DIR"

sudo mkdir -p $APP_DIR
sudo cp -r $WORKSPACE/app $APP_DIR/
sudo cp $WORKSPACE/app/requirements.txt $APP_DIR/

cd $APP_DIR
sudo python3 -m venv venv 2>/dev/null || true
sudo $APP_DIR/venv/bin/pip install -q -r $APP_DIR/requirements.txt

sudo cp $WORKSPACE/docs/devops-lab1.service /etc/systemd/system/devops-lab1.service
sudo systemctl daemon-reload
sudo systemctl enable devops-lab1
sudo systemctl restart devops-lab1

sleep 2
STATUS=$(curl -s -o /dev/null -w '%{http_code}' http://localhost:5000/health)
if [ "$STATUS" = '200' ]; then
echo 'Health check PASSED — HTTP 200'
else
echo "Health check FAILED — HTTP $STATUS" && exit 1
fi
```

6. systemd Service Unit

```
[Unit]

Description=DevOps Lab 1 Flask Application

After=network.target

[Service]

User=www-data

WorkingDirectory=/opt/devops-lab1

ExecStart=/opt/devops-lab1/venv/bin/python -m flask --app app.app run --host=0.0.0.0

Restart=always

Environment=FLASK_ENV=production

[Install]

WantedBy=multi-user.target
```

7. Build #2 — Automated Trigger Evidence

After pushing commit 7530c85 (bump app version to 1.0.1), Jenkins automatically detected the change via Poll SCM and triggered Build #2.

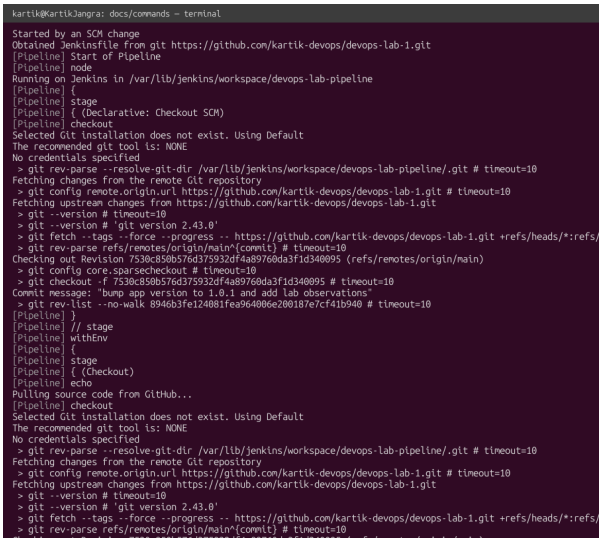


Fig: Build #2 — Checkout stage console output

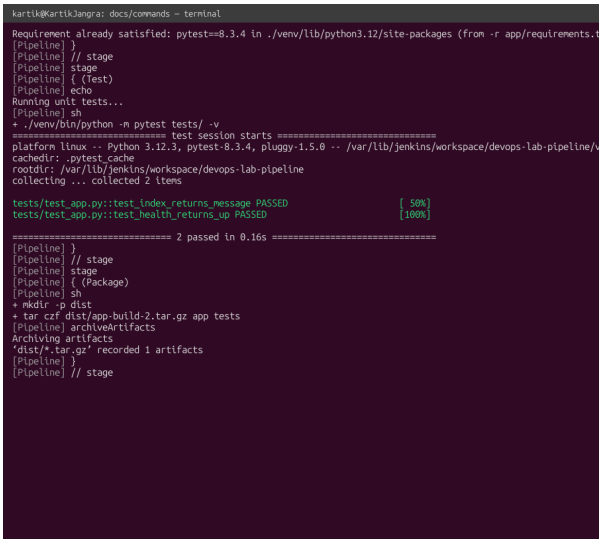


Fig: Build #2 — Build & Test stage

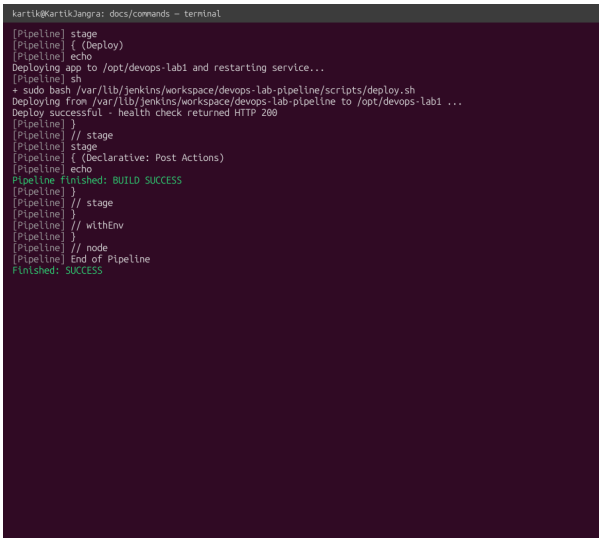


Fig: Build #2 — Package & Deploy stage



Fig: Build #2 — Pipeline SUCCESS



Fig: Git polling log showing automatic trigger

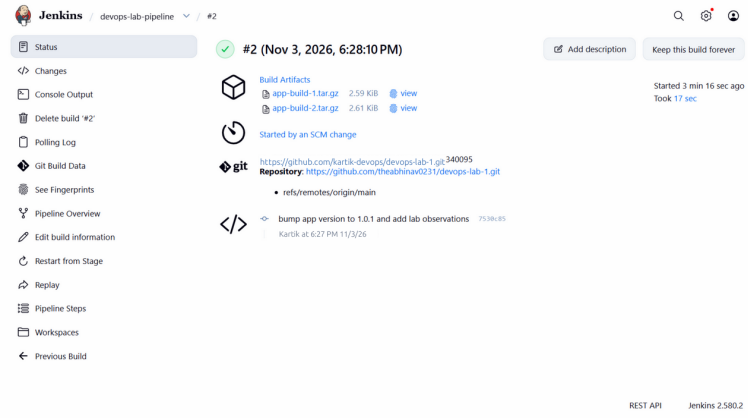


Fig: Build #2 pipeline overview

```
kartik@KartikJangra: docs/commands — terminal

kartik@KartikJangra:~$ sudo cp docs/devops-lab1.service /etc/systemd/system/
kartik@KartikJangra:~$ sudo systemctl daemon-reload
kartik@KartikJangra:~$ sudo systemctl enable --now devops-lab1

kartik@KartikJangra:~$ systemctl status devops-lab1 --no-pager
● devops-lab1.service - DevOps Lab 1 Flask Application
     Loaded: loaded (/etc/systemd/system/devops-lab1.service; disabled; preset: enabled)
     Active: active (running) since Tue 2026-11-03 18:03:37 UTC; 7min ago
   Main PID: 1629 (python)
      Tasks: 1 (limit: 9172)
     Memory: 19.4M (peak: 20.3M)
        CPU: 347ms
     CGroup: /system.slice/devops-lab1.service

kartik@KartikJangra:~$ curl http://localhost:5000/health
{"status":"UP","version":"1.0.0"}

kartik@KartikJangra:~$ cat /etc/sudoers.d/jenkins-deploy
jenkins ALL=(ALL) NOPASSWD: /usr/bin/bash /var/lib/jenkins/workspace/devops-lab-pipeline/scripts/deploy.sh
```

Fig: deploy.sh execution on deployment server

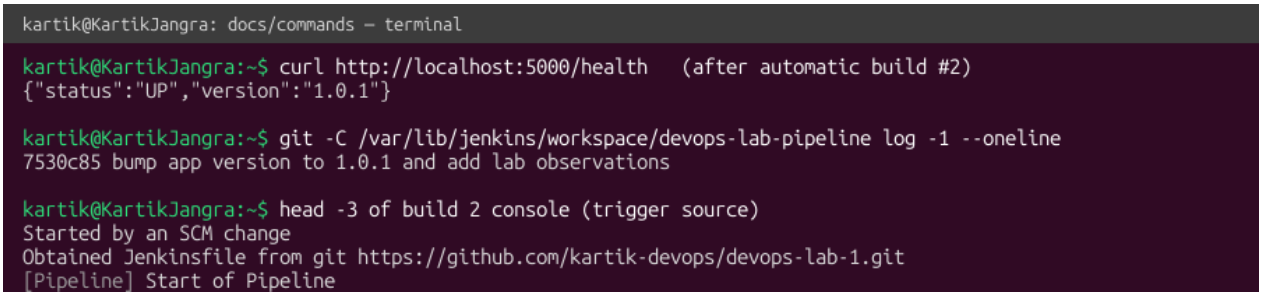


Fig: Service status verified — active (running)

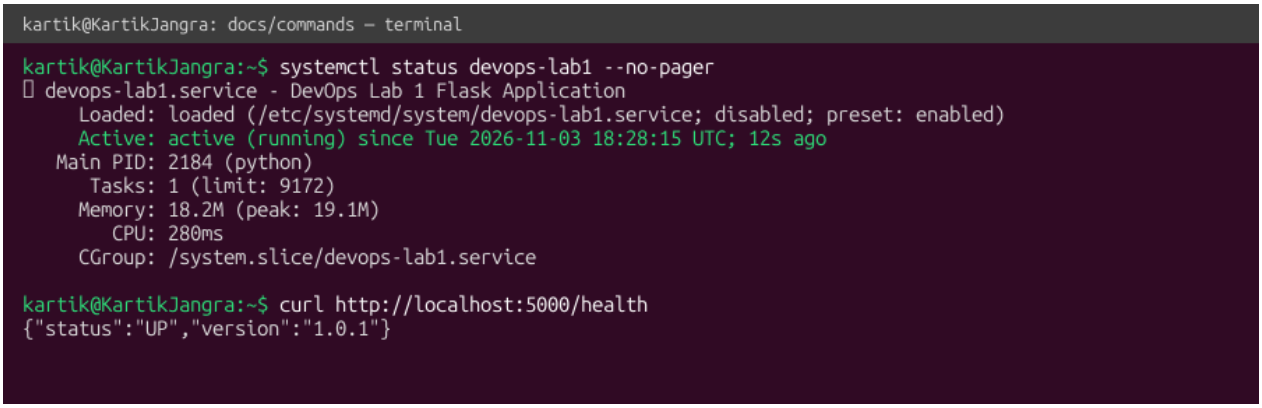


Fig: App status after full pipeline

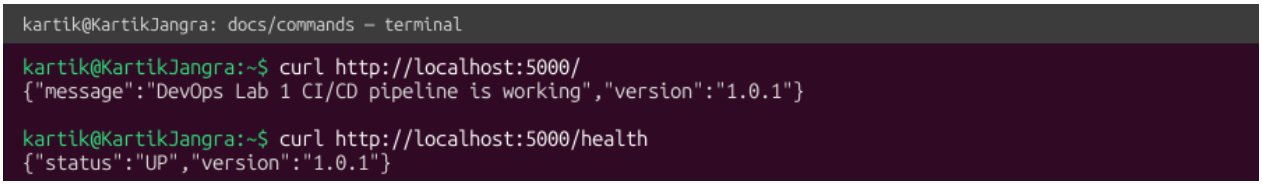


Fig: Flask /health endpoint — HTTP 200 response

8. Observations

- Jenkins detected the commit within 60 seconds and started Build #2 automatically.
- The deploy script copied application files, refreshed the virtualenv and restarted the service.
- The health check inside deploy.sh confirmed HTTP 200 from the /health endpoint.
- The systemd service remained active after deployment, confirming successful restart.

9. Conclusion

Full automated deployment was achieved. No manual intervention was required after the initial Jenkins job configuration. Every code commit triggered the pipeline, ensuring the latest version of the application was deployed and verified automatically.